THOMAS MORE

SMART HOME INTEGRATION IN EXPERIENCE LAB

Realization Document

Pierina Sabina Lopez Monserrate
Student Bachelor Applied Computer Science

2025 – 2026

Table of Contents

1. Introduction

This realization document presents and evaluates the final smart-home integration prototype developed during the internship at Thomas More Kempen – Research Group Mobilab & Care, in the Experience Lab. The document explains the purpose of the system, the reasoning behind the chosen technologies, the way the prototype was implemented, and the results of the final testing. The central claim of this work is that a locally controlled smart-home platform built on Home Assistant, integrated with the existing infrastructure of the Experience Lab, can provide a working, demonstrable, and reusable foundation for supporting safer night-time routines for older adults. The realised prototype confirms this claim in practice.

The project focuses on how smart-home technology can support older adults in living more safely, comfortably, and independently at home. More specifically, it addresses everyday situations such as night-time movement, bathroom-related safety, help activation, and simple routine control. Although the system was implemented in the Experience Lab, the environment should not be understood only as a technical lab setup. The apartment is important because it represents a realistic domestic setting that resembles the daily living context of an older person at home. This makes it possible to evaluate the prototype not only as a technical installation, but also as a practical support concept grounded in everyday situations.

The internship had two closely connected goals. The first was to deliver a functioning prototype that addresses a meaningful real-life problem: the safety of older adults during night-time movement at home. The second was to build that prototype in a way that can be reproduced, extended, and reused in future research within Mobilab & Care. These two goals shaped the technical choices documented in this report.

The project plan initially framed the work as a low-end and transferable smart-home concept. During the realisation, the system evolved into a hybrid integration. The apartment already contained a KNX-based domotics installation, a Nuki smart lock, and a Nobi fall-detection device. Rather than ignoring this infrastructure, the project built on it where relevant and combined it with new components, including Zigbee occupancy sensors, smart lighting through Zigbee2MQTT, a Raspberry Pi running Home Assistant, voice control through Alexa using a Matter Hub bridge, and Telegram and WhatsApp notification channels. The hybrid character of the final system should therefore be understood as a deliberate design choice based on the project context.

The realised system is a fully working local smart-home platform. Home Assistant runs on a Raspberry Pi and centralises the automation logic, entity management, and dashboard environment. It coordinates a Zigbee network of motion sensors, smart lights, and remote buttons; it integrates the Nuki smart lock as part of the assistance logic; and it exposes selected entities to Alexa through Home-Assistant-Matter-Hub. On top of this platform, the final prototype implements night path lighting, adaptive room-based lighting, weekday and weekend night-mode scheduling, a bathroom check-in flow, help-mode escalation with apartment-wide lighting response, and external notifications through Telegram and WhatsApp. In addition, a Python-based analysis component records night-mode usage and generates recommended activation and deactivation times based on a configurable learning window.

The remainder of this document is organised as follows. Chapter 2 presents the analysis and reasoning that supported the main design choices, including the platform, communication method, lighting strategy, and voice interface. Chapter 3 describes the technical realisation of the prototype, including the system architecture, integrations, configuration, and implemented logic. Chapter 4 reports on scenario-based testing in the Experience Lab and discusses the observed results, limitations, and supporting technical documentation. Finally, Chapter 5 concludes with a synthesis of the findings, the contribution of the work, and recommendations for future continuation within Mobilab & Care.

2. Analysis

This chapter presents the analysis that supported the design and technical realization of the smart-home prototype. The purpose of this analysis was not only to identify which technologies could be used, but also to determine which choices were the most appropriate for a realistic home-support system for older adults. In that sense, the chapter does not function as a general overview of available technologies, but as a reasoned justification of the final design decisions.

The analysis is grounded in two complementary perspectives. The first is the user and context perspective: the system had to address meaningful situations in the daily life of older adults living at home, especially situations related to safety, comfort, and independence. The second is the technical and practical perspective: the solution had to remain feasible, understandable, locally controllable, and sufficiently reusable for future continuation. Together, these perspectives shaped the final architecture, the selected platform, the communication strategy, and the additional voice and notification layers.

Accordingly, this chapter first discusses the problem context and design requirements, then analyses the selected platform and architecture, the communication and integration strategy, the role of voice interaction, and finally the relevance of lighting and adaptive support in the final concept.

2.1. Project context and design requirements

2.1.1. Supporting older adults at home

The starting point of this project is the concept of ageing in place, meaning the preference of many older adults to remain in their own home for as long as possible, provided that this can be done safely. Research shows that ageing in place is closely linked to independence, familiarity, autonomy, identity, and emotional continuity (Wiles et al., 2012). For that reason, support technology for older adults should not be designed only as technical infrastructure, but as something that fits into ordinary domestic life.

This is also why the project should not be framed mainly as a lab exercise. Although the prototype was implemented in the Experience Lab, the apartment is relevant because it resembles a realistic home environment in which every day routines can be simulated and evaluated. The goal is not to solve “lab problems,” but to explore how technology can respond to situations that older adults may face in their own homes.

2.1.2. Technology acceptance and accessibility

A second user-centred consideration is acceptance. Vaportzis et al. (2017) found that older adults are not necessarily resistant to technology, but often face barriers such as lack of confidence, fear of making mistakes, confusing interfaces, and the cognitive cost of learning new systems. The realised prototype responds to these barriers by relying on predictable triggers motion, time, a single physical button and by reserving complex configuration for the maintainer rather than the resident. The user-facing surface is intentionally minimal.

Accessibility extends beyond cognitive load. Voice interaction is included in the prototype not as a fallback for unreliable buttons, but as a complementary modality that addresses specific accessibility needs of the target user group: residents with reduced fine motor control, vision difficulties, or temporary mobility limitations may find it easier to speak to the system than to locate or press a physical control. The prototype therefore offers three coexisting input paths motion (passive), buttons (active, deterministic), and voice (active, hands-free) chosen so that no single user limitation removes the resident’s ability to operate the system.

2.1.3. Night-time movement as the main use case

A particularly important use case in this project is night-time movement, especially the path from the bedroom to the bathroom. This situation combines darkness, reduced balance, sleep inertia, and disorientation in a routine that may occur every night. For that reason, it is a small but highly relevant moment in which technology can provide clear support without becoming intrusive.

This focus is supported by research. Lu et al. (2022) showed that low-light conditions influence older adults' behaviour and fear of falling during night-time trips to the bathroom, and their findings support the usefulness of guidance-oriented lighting. Thölking et al. (2020) also found that guiding nightlights could reduce fear of falling and improve sleep-related outcomes in community-dwelling older adults. These studies justify a design that emphasizes calm, warm, low-level guidance lighting rather than harsh full-brightness activation.

The decision that follows from this is clear: the prototype should not try to automate the entire home equally. Instead, it should prioritise a small number of meaningful scenarios in which automation can genuinely improve safety and ease of use. Night path lighting, bathroom-related monitoring, and help escalation were therefore selected as the central functions of the final prototype.

2.1.4. Design requirements derived from the context

From this context, a number of design requirements emerged. The system had to be:

- non-invasive, meaning no unnecessary surveillance or complex medical monitoring;
- locally controllable, so that important functions do not depend entirely on cloud services;
- understandable, both for demonstration and for future reuse;
- flexible, because it had to combine existing infrastructure and newly added components;
- reproducible, so that future students or researchers can extend it;
- focused on meaningful routines, not on generic smart-home novelty.

These requirements directly shaped the later technical choices. Because the system had to remain understandable and flexible, a central local platform became necessary. Because the system had to support meaningful routines rather than one-off device actions, automation logic and helper states became essential. And because the design had to remain realistic for future implementation in other domestic settings, both cost and practical accessibility had to be considered in the comparison of tools and interfaces.

2.2. Platform and architecture choice

2.2.1. Why a central smart-home platform was necessary

Once the problem context had been defined, the next question was how the system should be organized technically. The prototype required a platform capable of integrating devices, running automations, visualizing system states, supporting dashboards, and remaining manageable as the implementation expanded. A fragmented approach, in which devices would remain divided across separate ecosystems, would have made the system harder to maintain, explain, and reuse.

For that reason, a central smart-home platform was required. The final decision was to use Home Assistant running locally on a Raspberry Pi 5. This choice followed directly from the design requirements described above. Home Assistant is relevant in this context because it combines broad integration support with strong local control and a transparent automation environment (Home Assistant, n.d.). This makes it suitable not only for implementation, but also for testing, documentation, and later handover. The Raspberry Pi 5 was selected as the host platform because it offers an affordable, compact, and well-documented solution for continuous local operation, and because it is widely used in practical smart-home setups, which improves reproducibility and maintainability.

The decision to prefer a local platform was also motivated by the nature of the use case. A system that supports safety-related routines, such as night path lighting or help escalation, benefits from predictable local behavior. Local control also keeps event history, entity states, and automations visible inside one platform, which is important both academically and practically.

2.2.2. Motivation of the platform comparison factors

To justify the platform choice more clearly, a weighted comparison was made. The decision factors were selected because they reflect the practical requirements of the project.

- **Integration flexibility:** important because the prototype combines sensors, lights, lock control, dashboards, notification channels, and voice interaction.
- **Local control:** important because core support behavior should remain available without depending completely on external cloud platforms.
- **Ease of prototyping:** important because the internship required a functional working prototype within limited time.
- **Maintainability / documentation:** important because the system should remain understandable and reusable after the internship.
- **Suitability for realistic home-support prototyping:** important because the chosen platform should work well in a domestic support context, not only in a purely technical demo.
- **Cost:** important because affordability matters for future reuse and because highly expensive solutions would weaken transferability.

These factors are not equally important. Integration flexibility and local control received the highest weight because they affect the viability of the whole system. Ease of prototyping and maintainability are slightly less critical but still central. Suitability for realistic home-support prototyping and cost remain relevant, but were given a slightly lower weight because they depend partly on the already-selected core platform requirements.

2.2.3. Weighted platform comparison

Weighted ranking matrix: central platform choice

Decision factor	Weight	Home Assistant	OpenHab	Proprietary ecosystem
Integration flexibility	4	5	4	2
Local control	4	5	5	2
Ease of prototyping	3	5	3	4
Maintainability / documentation	3	5	4	2
Suitability for living-lab demo	2	5	4	3
Cost	2	5	5	2
Total		76	66	36

This comparison supports the selection of Home Assistant as the central platform. It offered the strongest balance between integration flexibility, local control, implementation speed, and long-term maintainability. The decision also aligns with the final realised architecture, in which Home Assistant acts as the central orchestration layer for automations, helpers, dashboards, logging, and integration paths.

2.2.4. Architectural consequences of that decision

Choosing Home Assistant as the central platform also implied a specific architecture. The project would need:

- a main local controller;
- a communication layer for device events;
- a structure for automation logic and helper states;
- interfaces for dashboard, notifications, and voice interaction.

This architectural logic is reflected in the final realised system, in which Home Assistant coordinates Zigbee devices, MQTT-based communication, Nuki, Matter Hub, notification channels, and dashboard interfaces. The full system architecture is presented later in the technical realisation chapter.

2.3. Integration and communication strategy

2.3.1. Mixed-device communication as a design requirement

Because the prototype combines multiple types of devices and services, the communication strategy had to support a heterogeneous environment rather than a single ecosystem. The system includes wireless sensors, lights, plugs, manual control buttons, a smart lock, dashboard interfaces, and external messaging. This meant that a single-vendor setup would likely reduce flexibility.

The design decision that followed from this was to use a multi-vendor Zigbee strategy for the wireless device layer and a central message-based integration approach for device states and events. This kept the system open enough to support different brands while preserving a consistent logic layer in Home Assistant.

2.3.2. Why Zigbee2MQTT was selected

For the Zigbee layer, the final integration path was Zigbee2MQTT. This choice was made because Zigbee2MQTT is specifically intended for mixed Zigbee device environments and provides broad device support, visibility, and practical debugging possibilities (Zigbee2MQTT, n.d.).

This decision responds directly to the project requirements. Since the system had to remain flexible and reusable, it was more useful to adopt a multi-vendor approach than to depend on one restricted ecosystem. Zigbee2MQTT also fits well with Home Assistant because it exposes states and actions through MQTT, which makes the automation logic more transparent and traceable.

The decision therefore was not only “use Zigbee,” but more specifically: use Zigbee in a way that keeps the system flexible, inspectable, and expandable.

2.3.3. Why MQTT mattered

MQTT was selected as the communication layer between the Zigbee network and Home Assistant. The importance of MQTT in this project is both technical and practical. Technically, it is a lightweight and widely used publish-subscribe protocol. Practically, it makes device states, motion events, and actions visible as structured messages that can be logged, monitored, and reused in automation logic (Home Assistant, n.d.).

This is important in a prototype that needs to be explained and evaluated. MQTT does not merely “connect devices”; it helps structure the system in a way that makes triggers and responses more transparent. That transparency later supports both testing and documentation.

2.4. Voice interaction as an additional interface

2.4.1. Why voice was considered

Voice interaction was included in the analysis because it may offer an additional access path for certain users and certain situations. In a home-support context, voice can reduce physical effort and lower the barrier to triggering routines such as activating night mode, cancelling a check-in state, or asking for help. This can be especially useful for users who may find dashboards or manual controls less convenient.

At the same time, voice should not be treated as the baseline of the system. The core support logic must remain available through other control paths, including Home Assistant automations, dashboard controls, and physical buttons. Voice was therefore analysed as an additional layer of interaction, not as the only interface.

This distinction is important because it answers one of the central design questions of the project: how to add convenience without creating dependence. The design decision based on this reasoning was that voice control would be included only if it strengthened the system without becoming critical for safety functions.

2.4.2. Motivation of the voice comparison factors

A second weighted comparison was made to evaluate possible voice interfaces. The factors were chosen because they reflect the project's practical and contextual needs.

- **Practical prototyping speed:** important because the system had to be implemented within internship time.
- **Support for routines and prompts:** important because the project required routines such as "Welterusten," "Goedemorgen," and check-in related responses.
- Suitability for Dutch-speaking context: important because the prototype is used in a Dutch-speaking environment.
- **Cost:** important because solutions with recurring costs reduce long-term practicality.
- **Accessibility / availability:** important because the solution should be easy to obtain, configure, and use in a practical setup.
- **Future local / privacy potential:** important because local control remains relevant in a care-related context.

These factors are again weighted differently. Practical prototyping speed and routine support were prioritised because the system required a working proof of concept. Dutch-language suitability was also important because a voice interface that does not fit the local language context would be much less useful. Cost and accessibility were separated because a technology may be affordable but not practical to obtain or configure, or the reverse. Local/privacy potential was included as a forward-looking factor, even though it did not dominate the proof-of-concept phase.

2.4.3. Weighted voice interface comparison

Weighted ranking matrix: voice interface choice

Decision factor	Weight	Alexa	Google Assistant	Siri / HomePod	Home Assistant Voice
Practical prototyping speed	4	5	4	3	2
Support for routines and prompts	4	5	4	3	2
Suitability for Dutch-speaking context	3	4	4	4	2
Cost	2	4	4	2	3
Accessibility / Availability	2	4	4	2	3
Future local / privacy potential	2	2	2	3	5
Total		62	56	41	39

This comparison explains why Alexa was the most suitable voice layer for the proof of concept, even though other options remained possible for future iterations.

This comparison explains why Alexa was selected as the most suitable voice layer for the final prototype. It offered the strongest combination of routine support, rapid prototyping, availability, and Dutch-language suitability. Dutch support was also relevant in the local context and was confirmed by 2024 reporting on Alexa's Dutch-language availability (Tweakers, 2024).

2.4.4. Why the final implementation used Matter Hub

The final system did not expose Home Assistant entities to Alexa through the earlier Emulated Hue approach. Instead, it used Home-Assistant-Matter-Hub, which exposes selected Home Assistant entities as Matter devices to controllers such as Alexa (t0bst4r, 2026).

This decision followed from the broader requirement to keep the prototype as practical and sustainable as possible. Since the final prototype needed a working and acceptable route for Alexa integration, the Matter Hub bridge became the more suitable option in practice. It also fits the modular role of voice in the system: voice remains an additional interface layered on top of the local automation platform rather than the main controller of the system.

2.5. Lighting and adaptive behavior analysis

2.5.1. Why lighting is central in this prototype

Lighting was treated as a functional element of safety, not as decoration. Two principles guided the design. The first is gentle by default during the night, escalating only when needed: at night, lights come up at low brightness (15–25 %) and warm colour temperatures (2000–2400 K), so that walking from the bedroom to the bathroom does not trigger a jarring transition. Brightness rises to full only when help mode escalates. This aligns with the findings of Lu et al. (2022) and Thölking et al. (2020) that gentle guiding light is associated with reduced fear of falling and a better night experience.

The second principle is light as a routine signal. Constantino et al. (2025) discuss how the timing and spectral content of light interact with sleep and activity in older adults. Applied here, the system distinguishes between a calmer night state (warmer, dimmer light) and a normal day state (brighter, cooler light), so that the same lighting infrastructure supports both safety at night and orientation and comfort during the day.

2.5.2. From simple light control to routine support

The final lighting concept therefore goes beyond switching lights on and off. It distinguishes between:

- normal room lighting,
- night mode lighting,
- path guidance,
- check-in related states,
- help-mode escalation.

This means that the same lights can play different roles depending on system state. In practical terms, that supports a design in which the environment itself communicates part of the system logic. This makes the prototype easier to understand and more suitable for demonstration and reuse.

A further design choice is to treat night mode as a learnable state rather than a static toggle. Activation and deactivation events are logged with timestamps; a Python analysis component reads these logs over a configurable learning window (two days, one week, or one month) and writes recommended on/off times back to the Home Assistant helper entities used by the schedule. This goes beyond fixed scheduling: it allows the system to gradually align with the resident's actual rhythm.

2.6. Summary of analysis

The analysis showed that the most suitable basis for the prototype was a locally controlled smart-home platform built around Home Assistant and Raspberry Pi, combined with a multi-vendor Zigbee device layer, MQTT-based communication, adaptive lighting logic, and an additional voice layer through Alexa. These decisions were not driven only by technical convenience, but by the requirements of a realistic home-support context for older adults.

The analysis also clarified that the prototype should prioritize a limited number of meaningful routines over broad smart-home complexity. The main focus therefore remained on night-time movement, bathroom-related inactivity, help escalation, and understandable control through dashboard, buttons, and voice. This is consistent with both the research literature on ageing in place and the final realized structure of the system.

Finally, the analysis showed that a reusable support prototype requires more than isolated device selection. It requires a coherent architecture, justified decision criteria, and a balance between flexibility, local control, cost, accessibility, and practical implementation. These considerations shaped the final system that is described in the next chapter.

3. Technical Realization and Implementation

This chapter describes the final implemented smart-home prototype as it was realised at the end of the internship. The analysis in Chapter 2 identified the most suitable platform, communication strategy, and interaction model; this chapter explains how those choices were translated into a working system. The realised prototype is built around a local Home Assistant instance running on a Raspberry Pi and integrates Zigbee motion sensors, smart lighting, a Nuki smart lock, Telegram and WhatsApp notifications, and voice interaction through Alexa using a Matter Hub bridge.

The implementation was developed in the Experience Lab, but its relevance goes beyond that setting. The apartment is used here as a realistic domestic environment that resembles the daily living situation of an older person at home. For that reason, the technical realisation was not approached as a collection of isolated device tests, but as one coherent support system organised around meaningful routines such as night-time movement, bathroom-related safety, and help escalation.

The final prototype combines existing infrastructure already present in the apartment with newly added components and automations. This results in a hybrid architecture: the logic is centralised in Home Assistant, while selected existing devices, such as the Nuki lock, are integrated into that logic where they add practical value. At the end of the internship, the system operates as a finished and demonstrable prototype rather than as a partial proof of concept.

3.1. System architecture

The architecture of the realized system is hub-based. A Raspberry Pi running Home Assistant Operating System acts as the central controller and hosts the automation engine, helper entities, dashboard environment, and the main integration layers. Through this central role, Home Assistant manages the system state, receives sensor input, applies the automation logic, and controls the relevant outputs.

The final architecture combines five main layers:

1. a control layer, formed by Home Assistant on the Raspberry Pi;
2. an integration layer, including Mosquitto MQTT, Zigbee2MQTT, and Matter Hub;
3. an input layer, formed by motion sensors and Zigbee buttons;
4. an output layer, formed by lights, the Nuki smart lock, Alexa voice responses, and external notification channels;
5. an interaction layer, through which the system can be monitored and controlled from Home Assistant dashboards and personal devices such as a phone, tablet, or laptop.

This structure reflects the requirements discussed in Chapter 2. Because the prototype had to remain locally controllable, understandable, and reusable, the system was designed so that the essential automation paths remain inside one central platform. Motion detection, state changes, light control, lock control, and most routine logic are handled locally. External messaging and voice services extend the system, but they do not replace its core behavior.

The Experience Lab layout also influenced the architecture. The apartment is divided into meaningful zones for the implemented use cases: bedroom, hallway, bathroom, kitchen, and living room. These are not abstract technical labels, but domestic spaces through which a resident would move in daily life. The night-time path in particular crosses the bedroom, hallway, and bathroom, which means the system had to be designed around transitions between rooms rather than around one isolated sensor.

A final architectural characteristic is the hybrid integration of existing and newly added components. The project did not replace the apartment's infrastructure; instead, it built a new smart-home logic layer around Home Assistant and integrated existing elements where relevant. This applies most clearly to the Nuki smart lock, which is treated as a first-class actuator in the help-mode logic. The KNX system present in the

apartment is acknowledged as part of the existing infrastructure, but it is not used as the primary control path of the realized prototype.

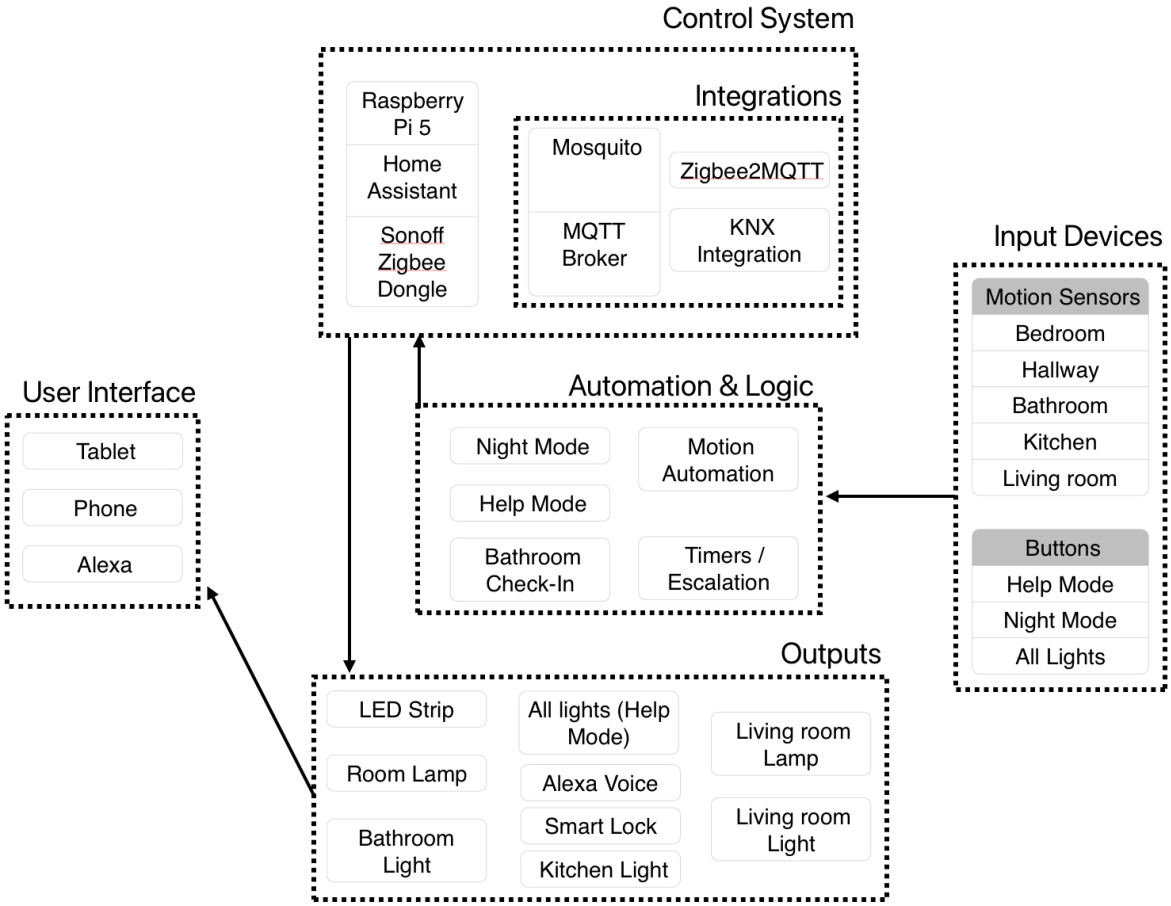


Figure 1. System architecture diagram

3.2. Hardware, devices, and cost

3.2.1. Core hardware and computing devices

The realised system uses one main Raspberry Pi as the smart-home server. This device runs Home Assistant Operating System and hosts the central platform, including the Zigbee2MQTT integration, Mosquitto MQTT broker, Matter Hub bridge, and the Python-based night-mode recommendation logic. During the internship, the configuration was migrated to a Raspberry Pi 5 with 8 GB of RAM, which provided the capacity needed for the final system.

The earlier touchscreen-based Raspberry Pi interface was retained only as an optional demonstration interface. It is useful in the Experience Lab because it allows a wall-mounted dashboard view during demonstrations, but it is not considered essential to the final concept. In a realistic home situation, the same interface can be accessed through devices that users already own, such as a smartphone, tablet, or laptop. For that reason, the touchscreen should be understood as an additional interface option rather than as a core requirement of the prototype.

3.2.2. Sensors, buttons, and actuators

The wireless device layer is built around Zigbee components. Five Zigbee motion sensors cover the bedroom, hallway, bathroom, kitchen, and living room. Their main role is to detect room occupancy and trigger the corresponding zone-based lighting or state transitions. Three Zigbee buttons provide manual control paths: one for night mode, one for help-related actions, and one for switching all lights at once.

The output layer consists primarily of lighting. The system includes a bedroom lamp, LED strip path lighting in the hallway, a bathroom light, kitchen light, living room light, and a smart plug controlling a floor lamp in the living room area. These actuators do not function merely as isolated lights; together they create the practical response layer of the system, supporting both normal room use and assistance-related states.

In addition to the newly added wireless devices, the prototype integrates the Nuki smart lock already present in the apartment. This lock plays an active role in the help-mode escalation logic and extends the system from environmental support toward access support.

3.2.3. Cost overview of added hardware

The hardware added during the project was deliberately selected from affordable off-the-shelf components. This reflects the broader project objective of keeping the system realistic as a reusable smart-home support concept.

The cost overview below includes the components added during the project. Existing infrastructure already present in the apartment, such as the KNX installation, the Nuki lock, and the Nobi device, is not included in the totals.

Product	**Quantity**	**Unit price (€)**	**Total (€)**	**Supplier**
Ceiling light for toilet	1	39,99	39,99	Amazon
Motion sensors	5	17	85	Amazon
Smart plug	1	14,9	14,9	Amazon
Lamp bulbs	3	23,6	70,8	Amazon
LED strips	2	34,99	69,98	Amazon
Zigbee coordinator	1	22,84	22,84	Amazon
Amazon Echo Spot	1	94,99	94,99	Amazon
Diffuser channel	1	22,8	22,8	Amazon
Room lamp	1	9,99	9,99	IKEA
Living room and kitchen cord lamps	2	3,99	7,98	IKEA
Living room and kitchen shade lamps	2	8	16	IKEA
Button set	1	57	57	Amazon
Raspberry Pi	1	170	170	Amazon
Subtotal main hardware	**682,27**			

This overview shows that the largest cost items are the computing hardware and the voice device, while the sensor and lighting layer remains relatively affordable. This supports the design claim made earlier in the document: the realised system is not cost-free, but it remains based on accessible consumer components rather than specialist care technology.

3.3. Home Assistant platform and state model

The whole system is organised around Home Assistant as the central logic layer. Its importance in the final implementation lies not only in device integration, but also in the fact that it hosts the state model of the system. Instead of each device acting independently, the prototype uses a small set of helper entities to represent the current condition of the environment and to coordinate the automation behaviour across rooms.

The most important helpers are the booleans for night mode, bathroom session, bathroom check-in, help mode, and test mode. Together, these define whether the system is in normal operation, in a bathroom-related monitoring state, in escalation, or in a shortened demonstration mode. Additional helpers store weekday and weekend schedule times, automatic night-mode settings, and night-mode recommendation values derived from the Python analysis component.

This state model is intentionally compact. It allows the system to represent the main functional situations clearly without becoming overloaded with intermediate flags. That decision improves readability in the configuration and makes the automation logic easier to follow during testing and maintenance.

A particularly practical helper in the final implementation is test mode. When activated, it shortens the waiting times in all relevant automations from minutes to a few seconds. This made it possible to validate and demonstrate timing-dependent behaviours, such as bathroom inactivity and help escalation, without waiting through the full real-life intervals.

Helper entity	Type	Purpose
input_boolean.night_mode	boolean	Master flag for the night character of all automations
input_boolean.bathroom_session	boolean	True while a bathroom visit is in progress
input_boolean.bathroom_check_in	boolean	True during the safety check-in window
input_boolean.help_mode	boolean	True during active help-mode escalation
input_boolean.test_mode	boolean	When on, all wait timers are shortened to seconds
input_boolean.night_mode_auto_enabled	boolean	Master switch for the automatic night-mode schedule
input_datetime.night_mode_active_weekday_time	datetime	Weekday auto-on time for night mode
input_datetime.night_mode_active_weekday_off_time	datetime	Weekday auto-off time
input_datetime.night_mode_active_weekend_time	datetime	Weekend auto-on time
input_datetime.night_mode_active_weekend_off_time	datetime	Weekend auto-off time
input_select.night_mode_learning_window	select	Adaptive analysis window: 2 days / 1 week / 1 month
input_button.reset	button	Triggers a full system reset

3.4. Integrations and connected devices

The realised system depends on the coordinated use of several integrations, each with a specific role in the final architecture.

3.4.1. Zigbee2MQTT and MQTT communication

The wireless device layer is integrated through Zigbee2MQTT, connected to a USB Zigbee coordinator. This makes it possible to pair Zigbee sensors, buttons, and lights from different brands and expose them to Home Assistant as usable entities. The communication between Zigbee2MQTT and Home Assistant runs through MQTT, which functions as the message layer for device states, triggers, and actions.

This integration path proved suitable for the final prototype because it provides strong compatibility for mixed-device environments and makes the device network visible and traceable. Sensor states, occupancy events, and button actions are all available in Home Assistant and can therefore be reused directly in the automations.

3.4.2. Nuki smart lock integration

The Nuki smart lock was integrated as part of the final system and functions reliably in the finished prototype. Rather than remaining only as a connected accessory, it was incorporated into the assistance logic of the prototype. In help mode, the lock can be opened automatically so that a responder can enter the apartment without delay.

This is an important part of the final implementation because it extends the system beyond light control and environmental automation. It demonstrates that the prototype can connect support-related routines not only to visibility and orientation, but also to access management.

3.4.3. Telegram and WhatsApp notifications

The prototype also implements external notification channels for support-related scenarios. Telegram was configured through the Home Assistant Telegram integration so that assistance messages can be sent to predefined chat destinations. WhatsApp notifications were configured through CallMeBot as a second notification path. In the final prototype, these notification channels are used in help mode to send clear text-based alerts outside the local Home Assistant environment.

This dual-channel approach increases redundancy. It means that help-related alerts do not remain limited to the dashboard inside the apartment, but can also reach external recipients through commonly used messaging platforms.

3.4.4. Voice interaction through Alexa and Matter Hub

Voice interaction is implemented in the final system through Alexa using Home-Assistant-Matter-Hub. This bridge publishes selected Home Assistant entities as Matter devices on the local network, making them available to Alexa for discovery and control. In practice, this means that night mode, help-related states, and selected routine entities can be triggered through voice in the same way as through the dashboard or buttons.

In the final implementation, voice is treated as an additional interface rather than the primary one. The core logic remains in Home Assistant, and every critical routine remains accessible through non-voice control paths as well. This reflects the design principle established earlier in the analysis: voice can increase accessibility and convenience, but the system should not depend on it alone.

3.5. Implemented automations

Twenty-six automations are running in the prototype. To make this large set readable, they are first introduced in terms of the five core scenarios they collectively support, and then described group by group with concrete behaviour drawn from the live automation definitions. The five core scenarios each tested in Chapter 4 are:

- Night-time bathroom trip: guided low-level lighting from bedroom to bathroom and back when the resident gets up at night.
- No-return / fall escalation: if the resident does not return from the bathroom, the system raises a safety check-in and, if unanswered, escalates to help mode (lights up, lock open, Telegram + WhatsApp).
- Daytime adaptive lighting: bright, cool, task-oriented lighting that follows the resident through the apartment during the day.
- Real-time night-mode transition: when night mode toggles, currently lit rooms smoothly settle into the appropriate character.
- Adaptive learning over time: the night-mode schedule converges to the resident's actual rhythm based on observed activation history.

These scenarios are realised by six functional groups of automations described next.

3.5.1. Night mode scheduling (manual and automatic)

Night mode functions as a master state that determines how the apartment behaves during night-time use. It can be activated manually through the dashboard, through a Zigbee button, or automatically according to weekday and weekend schedules. The distinction between weekday and weekend timings makes the system more realistic because household routines are not always identical across the full week.

3.5.2. Room-based adaptive lighting

Five zones are implemented with room-based lighting logic: bedroom, hallway, bathroom, living room, and kitchen. Each zone reacts to occupancy in a way that depends on whether night mode is active. During daytime use, rooms use brighter and cooler light settings. During night mode, the same lights respond with warmer and dimmer settings that are more appropriate for quiet movement and reduced disruption.

Zone	Sensor entity (suffix)	Day character	Night character	Idle timeout
Bedroom	Sensor_one_occupancy	(no auto-on; manual)	20 % / 2000 K	1 min
Hallway	Sensor_two_occupancy	(LED strip, off)	20 % LED strip	30 s (path-wide)
Bathroom	Sensor_three_occupancy	90 % / 4000 K	20 % / 2200 K	3 min
Living room	Sensor_four_occupancy	80 % / 3500 K	15 % / 2200 K	30 min
Kitchen	Sensor_five_occupancy	90 % / 4000 K	25 % / 2400 K	10 min

Four of the five zones therefore have a night character (warmer and dimmer) and a day character (cooler and brighter). The hallway is special: it has only one mode because its sole purpose is to act as a guiding strip during the night path. The kitchen automation additionally turns on switch.woonkamerlamp, extending the lit area into the adjacent corner of the living room during cooking. The light-off timers were chosen to match how each zone is used; they all collapse to five seconds when test_mode is on, which made demonstration practical.

3.5.3. Bathroom session and check-in flow

The bathroom is the safety focus of the system. When the bathroom occupancy sensor triggers, the Badkamer – licht adaptief aan automation does three things at once: it turns on the bathroom light at the appropriate level for the current mode, it raises bathroom_session to mark that the resident is in the bathroom, and it clears any prior bathroom_check_in and help_mode flags so that a new visit always starts from a clean state.

If bathroom motion stops while bathroom_session is still active, a separate automation (Check-in – inschakelen bij geen badkamerbeweging) starts a five-minute countdown. If the timer elapses with no further motion, the system raises the bathroom light to 100 % and sets bathroom_check_in. This is the Veiligheidscontrole state visible on the dashboard. Two paths can clear it: the resident moving (bathroom motion or hallway motion) or pressing the help button. If neither happens within one further minute, a downstream automation (Helpmodus – activeren na check-in) raises help_mode.

A cleanup automation (Check-in uit bij gangbeweging) handles the case where the resident leaves the bathroom and walks back through the hallway: it clears bathroom_session, bathroom_check_in, and help_mode together, so the system returns to its idle night-mode configuration without leaving stale state behind.

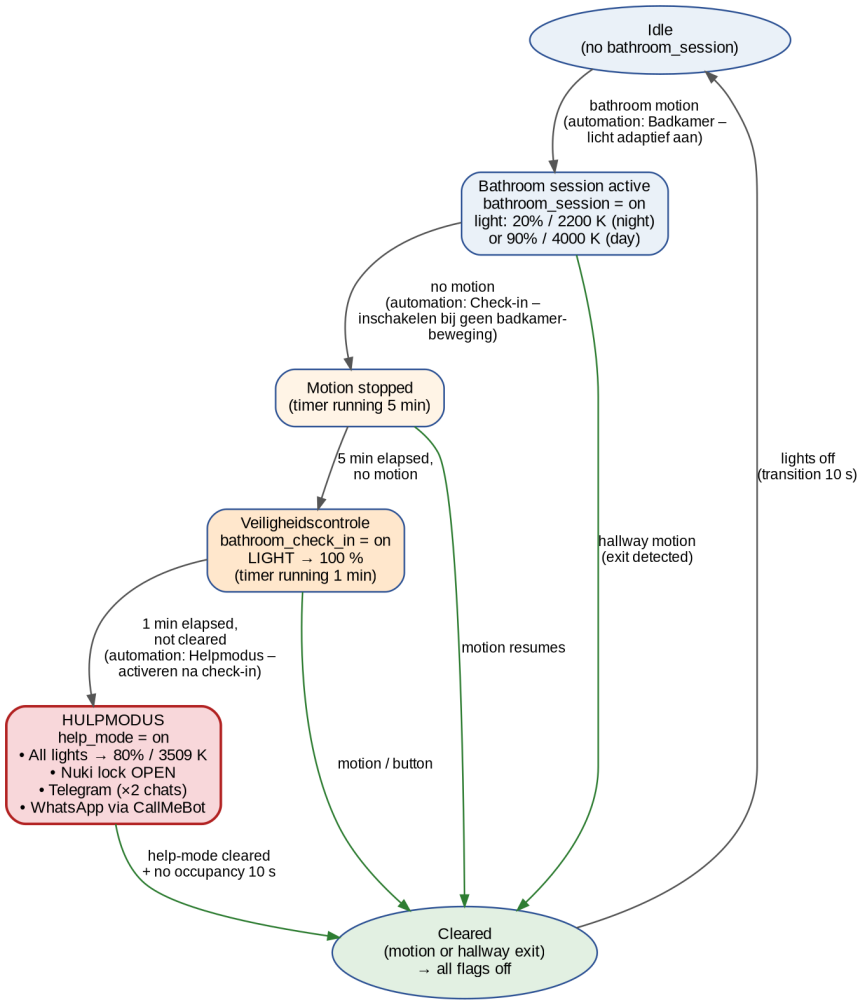


Figure 2. Bathroom session and help-mode escalation. Black arrows are the escalation path; green arrows are the recovery paths that return the system to idle without an alert.

3.5.4. Help mode and escalation behaviour

When help_mode becomes on, the Hulpmodus is geactiveerd automation performs four actions in parallel:

1. All five lights are set to 80 % brightness at 3509 K with a 10-second transition, ensuring the apartment is fully and warmly lit.
2. The Nuki smart lock is sent an open action, so a responder can enter without delay.
3. A Telegram message is dispatched to both notify.xlab_xlab with the title Hulpmodus is geactiveerd and a short body explaining that the user may need help.
4. A WhatsApp message with the same content is dispatched through notify.

When help_mode is later cleared, the Hulpmodus uit – verlichting resetten automation waits ten seconds and verifies that all four motion sensors are clear; only then does it turn the lights off again, so the apartment does not go dark while a responder is still inside.

3.5.5. Real-time response to night mode changes

A potential problem in any night-mode system is that toggling the mode while lights are already on creates a mismatch: the bathroom light could remain at 90 % even though night mode has just been activated. The Nachtmodus – actieve lichten aanpassen automation solves this. It triggers on every state change of input_boolean.night_mode, walks through the four main lights (light.bathroom_light, light.kitchen_light, light.living_room_light, light.roomlamp), and for each one that is currently on, applies the new mode's brightness and colour temperature with a 10-second transition. The result is a smooth, immediate visual change that makes the mode transition feel intentional rather than abrupt.

3.5.6. Manual overrides and system reset

Three Zigbee buttons provide manual override paths. Knop – nachtmodus schakelen toggles night mode. Knop – helpmodus schakelen toggles bathroom_check_in and help_mode together, providing both an early "I'm fine" cancel and a panic-button equivalent. Alle lichten schakelen toggles all four main lights at once, picking either the night character (20 % at 2200 K) or the day character (80 % at 2200 K) depending on the current mode. A dashboard button (input_button.reset) drives the Systeem – reset automation, which clears all four state booleans and turns off every actuator. This is the maintainer's panic button during demonstrations.

3.6. Night mode recommendation logic

Beyond the manual and scheduled toggling, the prototype includes an adaptive component that lets the night-mode schedule converge over time to the resident's actual rhythm. This is currently the only part of the system that learns from history; other behaviors remain deterministic by design, because predictability matters for safety-relevant automations. The implementation has three pieces.

3.6.1. What is recorded

Every change of input_boolean.night_mode produces a state-transition record in Home Assistant's history database. The Python analysis component, running alongside Home Assistant, queries this history through the local Home Assistant API and collects the timestamps of on and off transitions over the configured learning window. No additional logging infrastructure is required: the analysis loop reuses Home Assistant's built-in event history.

3.6.2. Python analysis component

The component runs on an event-driven schedule. Three trigger automations, Nachtmodus analyse elke 2 dagen, …elke week, and …elke maand, fire a custom Home Assistant event (run_night_mode_analysis) at 03:00, 03:05, or 03:10, depending on which learning window is currently selected via input_select.night_mode_learning_window. The Python component listens for that event, fetches the relevant history slice, computes a representative on time and off time for weekdays and a separate one for weekends (using a robust statistic so that one anomalous night does not skew the schedule), and writes the four results back to the corresponding input_datetime helpers. The next scheduled night-mode automation then runs at the updated times automatically.

3.6.3. Learning windows

Three learning windows are available. Two days makes the schedule extremely responsive, suitable for an initial calibration period when the resident's actual rhythm is unknown. One week is the operational default: it averages over a full weekly cycle and is robust to single nights when, for example, the resident went to bed unusually late. One month is intended for stable long-term operation, where the schedule should track gradual drifts (seasonal changes, daylight-saving transitions) without reacting to short-term noise.

3.7. Dashboard and monitoring approach

The Home Assistant dashboard is an important part of the realised prototype. Its role is broader than simple user interaction. In the final system, the dashboard functions as a user interface, a monitoring surface, a testing tool, and a demonstration tool. This is important because the prototype is not only meant to run automations in the background, but also to remain understandable and controllable by future users, researchers, and demonstrators.

The dashboard was therefore organised into four main views, each with a distinct role in the system. This structure improves readability and prevents all controls from being grouped into one overcrowded screen. It also responds directly to the different needs of the prototype: daily control, recommendation handling, testing, and area-based inspection.

3.7.1. Thuis view

The Thuis view is the main control view of the system. It is the most suitable page for normal day-to-day interaction because it brings together the most relevant controls and status indicators in one place. This includes direct control of the main lights, the Night Mode, Check-in, and Help Mode states, the door lock, the reset button, and the indicators related to automatic night mode and available recommendations.

This view is the most user-oriented part of the dashboard. It is intended to provide a clear overview without requiring the user to navigate through technical details. For that reason, it is the most suitable page for ordinary interaction with the prototype and the most representative interface for demonstration of the main system behaviour.

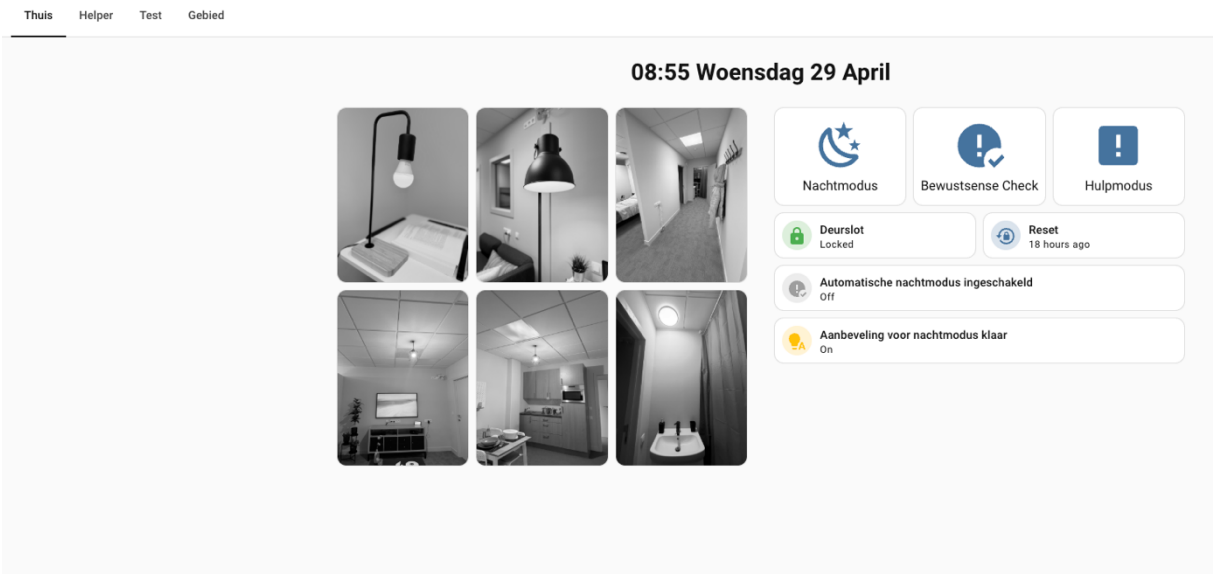


Figure 3. Thuis view

3.7.2. Helper view

The Helper view is dedicated to the night-mode recommendation logic. Its purpose is not general system control, but the management of the recommendations generated by the Python analysis component described in Section 3.6. In this view, the user or demonstrator can read the recommendation text, accept or reject the proposed night-mode schedule, select the learning window, manually trigger the analysis, and inspect the suggested weekday and weekend activation and deactivation times.

This view is important because it makes the recommendation system visible and transparent. Instead of adjusting schedules silently in the background, the prototype presents the proposed changes explicitly. In this way, the adaptive behaviour remains inspectable and controllable rather than opaque.

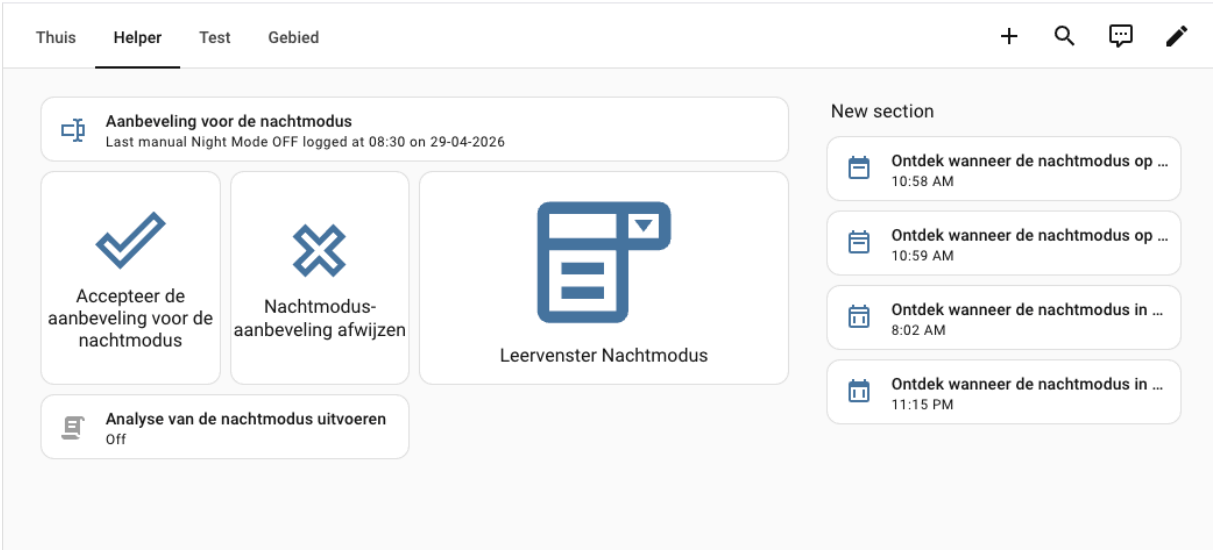


Figure 4. Helper view

3.7.3. Plan view

The Plan view is primarily intended for testing and demonstration. It presents a simplified floor-plan representation of the apartment and shows the main sensors, lights, and control states in a spatial layout. This makes it easier to observe how the prototype behaves across the different zones of the apartment. In this view, motion activity, light responses, lock state, and key control modes such as Night Mode, Help Mode, Check-in, and Test Mode can be monitored and triggered directly.

This view is especially useful during validation because it allows the behaviour of the system to be followed visually in relation to the apartment layout. Instead of checking entities one by one, the demonstrator can see how motion in one room leads to a lighting response or state change in another part of the apartment. It is therefore the most suitable dashboard page for explaining the spatial logic of the prototype during demonstrations.

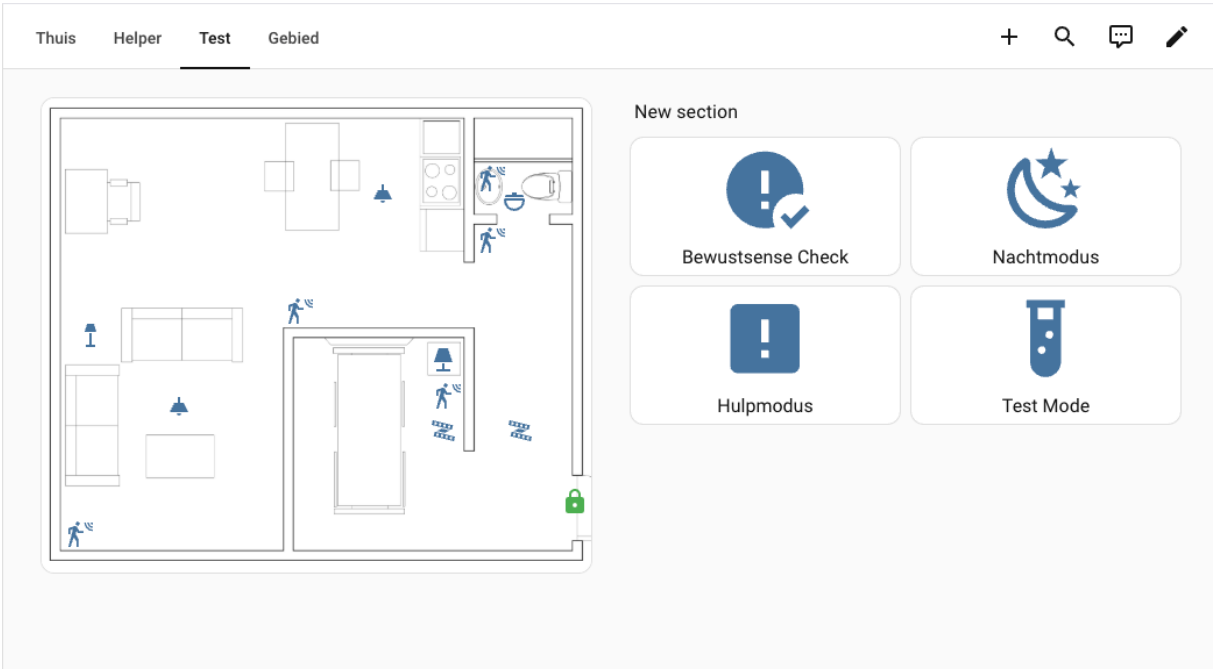


Figure 5. Plan view

3.7.4. Gebied view

The Gebied view is the area-based dashboard page. It groups the entities according to the apartment zones, such as hallway, bedroom, bathroom, kitchen, and living room. This view is useful for structured inspection of devices by room and makes it easier to check whether the relevant sensors, lights, and states in a given area are functioning correctly.

Compared with the Thuis view, the Gebied view is more technical and more suitable for maintenance or inspection. It is less focused on immediate control and more on organising the system according to the physical environment. This is particularly helpful during testing, because it allows the evaluator to verify device behaviour room by room.

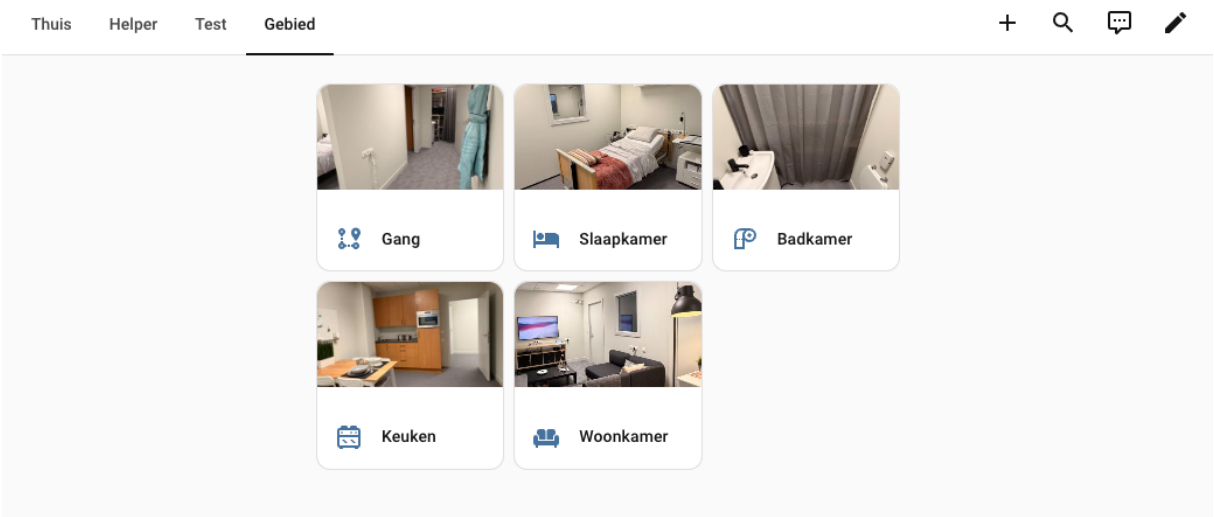


Figure 6. Gebied view

3.7.5. Monitoring value of the dashboard

Taken together, these four views make the dashboard a central part of the realised prototype. The Thuis view supports everyday control, the Helper view supports recommendation management, the Plan view supports spatial testing and demonstration, and the Gebied view supports room-based inspection. This layered dashboard structure improves the transparency of the system and helps different users interact with it according to their role.

From a monitoring perspective, the dashboard works together with the Home Assistant event history and logs. The visual interface shows the current system state, while the history and logs make it possible to trace state changes, motion detection, light responses, and escalation events over time. This combination is valuable because a demonstrable smart-home prototype should not only function, but should also remain explainable and verifiable.

4. Testing, Limitations, and Documentation

This chapter reports on how the prototype was tested in the Experience Lab and what the tests showed. The five scenarios introduced at the start of section 3.5 are now described in detail: each is presented as a real walkthrough performed in the lab, with the underlying automation flow, the observed behaviour, and a discussion of the result. Voice control through Alexa is treated as a first-class input channel in this chapter, alongside motion and physical buttons; section 4.2 lists the six Alexa routines that were configured for the prototype, and the scenario walkthroughs in section 4.3 exercise the routines in context. The chapter then summarises the encountered limitations and how they were resolved or scoped, and closes with a brief note on the documentation deliverables.

4.1. Testing approach

Three complementary testing methods were used. The first was unit-level verification of each automation: triggering it manually with the relevant entity in the Developer Tools and confirming that the expected actions ran. The second was scenario-level verification in the apartment: walking through the five scenarios below and observing the system's response. The third was long-running verification over multiple nights, with the system in full operation and the logbook reviewed afterwards to confirm that the night-mode schedule, the adaptive learning component, and the various timeout-driven cleanup automations behaved as designed.

Throughout the testing phase, three input channels were exercised in parallel: motion sensors as the passive input path, the three Zigbee buttons as the deterministic active path, and Alexa as the hands-free active path. This mirrors the analysis chapter's framing of voice as an accessibility complement rather than as a fallback (section 2.1.2): testing across all three paths confirms that no single user limitation removes the resident's ability to operate the system.

The input_boolean.test_mode helper was used to compress wait timers from minutes to seconds. This is essential: the bathroom check-in normally waits five minutes, and the help-mode escalation adds a further minute. Without test mode, validating a single end-to-end run of the safety flow would take six minutes; with test mode on, the same sequence takes about fifteen seconds.

4.2. Configured Alexa routines

Six Alexa routines were configured in the Alexa app and bound to Home Assistant entities through the Matter Hub bridge described in section 3.4.4. Each routine maps directly to a state change of one or more of the helper booleans introduced in section 3.3. The routines are listed below; their behaviour is exercised within the scenario walkthroughs that follow.

Routine name	Trigger phrase / event	Status	Action on Home Assistant state
Welterusten	“Alexa, welterusten” (and one alternative phrase)	Enabled	Speaks “Good night. Talk to you tomorrow!”, then sets night_mode → on
Goedemorgen	“Alexa, goedemorgen” (and one alternative phrase)	Enabled	Sets night_mode → off
Hulpmodus	“Alexa, hulpmodus” (and one alternative phrase)	Enabled	Sets help_mode → on (immediate escalation, bypasses the timer chain)
stop hulp	“Alexa, stop hulp” (and one alternative phrase)	Enabled	Sets help_mode → off
alles goed	“Alexa, alles goed” (and one alternative phrase)	Enabled	Speaks “A-OK”, sets help_mode → off, sets bathroom_check_in → off
Bewustsense check	Event: bathroom_check_in turns on	Enabled	Plays a short song on the Alexa device as an audible nudge

The first five routines are voice-triggered and are currently enabled in production; together they let the resident control night mode, escalate to help mode, and cancel a false alarm without touching a button or a screen. The sixth routine, Bewustsense check, is event-triggered: when bathroom_check_in turns on (the Veiligheidscontrole state on the dashboard), Alexa would play a short song as an audible nudge before help mode escalates. The underlying mechanism, exposing the Veiligheidscontrole flag to Alexa as a Matter device, which Alexa then observes as a state-change trigger has been verified to work.

4.3. Scenario-based validation

To make the scenarios concrete, they are described in terms of a simulated resident named Anna. Each scenario is presented in the same structure: narrative, automation flow, observed behaviour, and result and discussion.

4.3.1. Scenario A: Anna's night-time bathroom trip

Narrative. At 22:00, instead of pressing the night-mode button, Anna says "Alexa, welterusten." Alexa replies "Good night. Talk to you tomorrow!", and night_mode is set to on. At 03:00 she gets up to use the bathroom. At 06:30 she wakes and says "Alexa, goedemorgen," which sets night_mode back to off.

Automation flow. The Welterusten routine sets the Nachtmodus Matter switch to on. On the Home Assistant side, the same toggle is input_boolean.night_mode, so the rest of the night-time flow is identical to the schedule-driven path. Bedroom motion later fires binary_sensor.occupancy to on; with night_mode = on, the "Slaapkamer – beweging nachtmodus aan" automation turns on the bedroom roomlamp at 20 % / 2000 K and the corridor LED strip at 20 %, both with a 5-second transition. As Anna enters the hallway, "Gang – beweging nachtpad aan" reinforces the LED strip at 20 %. The bathroom sensor triggers "Badkamer – licht adaptief aan", which turns on light.bathroom_light at 20 % / 2200 K, raises bathroom_session, and clears any prior check-in flags. Anna spends about four minutes in the bathroom and walks back through the hallway and into the bedroom. The "Nachtpad – lichten uit" automation waits 30 seconds after every sensor has cleared, then fades all night-path lights off with a 10-second transition. The bedroom roomlamp turns off after a one-minute idle period via "Slaapkamer – licht uit na geen beweging". When Anna says "Alexa, goedemorgen" at 06:30, the Goedemorgen routine sets night_mode to off; this triggers the real-time light-adjustment automation discussed in Scenario D.

Observed behaviour. The voice trigger felt indistinguishable from a schedule trigger from the resident's point of view. The verbal acknowledgement ("Good night. Talk to you tomorrow!") provided useful confirmation that the routine had run before Anna stopped paying attention. The corridor LED strip and bedroom roomlamp turned off as expected after the bathroom trip. The morning Goedemorgen routine completed silently in under a second; brevity at 06:30 was deliberate.

Result and discussion. The voice path integrates seamlessly because Welterusten and Goedemorgen converge on the same input_boolean.night_mode that the schedule and physical button drive. The verbal acknowledgement on Welterusten is a small but important detail: smart-home commands that fail silently are notoriously confusing for older adults, and a short spoken reply closes that feedback loop. The voice trigger also sits earlier in the resident's evening routine than a button press tends to, which means more nights begin with night_mode already on by the time the resident actually walks toward the bedroom, an arguably better user experience than relying on a fixed clock-time schedule.

4.3.2. Scenario B: No-return escalation

Narrative. This is the safety-critical scenario the system is built around. Anna goes to the bathroom at 03:00 and, partway through, becomes unable to leave, for example, after a fall or after feeling unwell. Three voice paths are tested in addition to the timer-driven escalation: an explicit early escalation ("Alexa, hulpmodus"), an explicit recovery ("Alexa, alles goed"), and a post-escalation cancel ("Alexa, stop hulp").

Automation flow. timer-driven path. Anna's entry into the bathroom raises bathroom_session as in Scenario A. After a few minutes she stops moving. The "Check-in – inschakelen bij geen badkamerbeweging" automation, having seen bathroom_session = on and the bathroom sensor go to off, starts a five-minute timer. When the timer elapses, it raises bathroom_check_in and lifts light.bathroom_light to 100 %. This is the "Veiligheidscontrole" signal: a deliberate visual nudge giving the resident an opportunity to either move or to cancel. If neither happens within one further minute, "Helpmodus – activeren na check-in" raises help_mode, and the "Hulpmodus is geactiveerd" automation runs in parallel:

every light in the apartment ramps to 80 % at 3509 K over 10 seconds; the Nuki smart lock receives an open action; a Telegram message is dispatched to notify.xlab_xlab; and a WhatsApp message is dispatched through notify.whatsapp_mobilab.

Automation flow. voice paths. At any point during the timer chain, Anna can speak. Saying “Alexa, alles goed” triggers the alles goed routine: Alexa replies “A-OK” and turns off both Hulpmodus and Veiligheidscontrole, which clears help_mode and bathroom_check_in in a single phrase. This duplicates what hallway motion would do “Check-in uit bij gangbeweging” but is reachable by a resident who cannot stand. Saying “Alexa, hulpmodus” instead triggers the Hulpmodus routine, which sets help_mode to on directly, bypassing the wait timers entirely; the same “Hulpmodus is geactiveerd” cascade then runs (lights, lock, Telegram, WhatsApp). After help mode has fired, “Alexa, stop hulp” triggers the stop hulp routine, which sets help_mode to off; the “Hulpmodus uit – verlichting resetten” automation then waits 10 seconds for occupancy clearance before turning the lights off. The Bewustsense check routine, when enabled, would additionally play a song on the Alexa device the moment bathroom_check_in turns on, sitting between the visual nudge (light to 100 %) and the full help-mode cascade.

Observed behaviour. timer-driven path. With test_mode = on, the full chain, bathroom entry, motion absence, check-in, help mode, completed in roughly twenty seconds. Telegram delivery to both configured chat IDs was reliable and arrived within one to two seconds. The WhatsApp message via CallMeBot arrived a few seconds later. The Nuki lock opened on the same automation run, with no audible conflict between the lock action and the lights. With test_mode = off, the same chain completed in six minutes total, exactly matching the configured timers.

Observed behaviour. voice paths. Saying “Alexa, alles goed” during the check-in window produced Alexa’s “A-OK” reply within roughly one second; both flags cleared in the next automation cycle and the bathroom light returned to its night character. Saying “Alexa, hulpmodus” bypassed the timer chain entirely: within one to two seconds of the spoken phrase, help_mode was on and the full cascade ran (lights, lock, Telegram, WhatsApp). “Alexa, stop hulp,” spoken after help mode had fired, cleared help_mode equally fast, and the “Hulpmodus uit – verlichting resetten” automation took the apartment back to its idle state once occupancy had cleared.

Result and discussion. The voice paths transform the help-mode flow from a one-direction escalation into a two-way conversation. A button-only system forces the resident to reach a physical control to either escalate or cancel; with Alexa configured, “Alexa, alles goed” is reachable from anywhere in the apartment, which matters most precisely when the resident has fallen and cannot reach the help button. The “Alexa, hulpmodus” path is equally important on the other side: a resident who feels unwell does not have to wait six minutes for the timer chain to escalate on her behalf, she can request help immediately. The pairing of the two recovery routines, alles goed for the resident, stop hulp for a responder once they are inside, keeps both intuitive without overloading either phrase. The Bewustsense check routine, will close a useful gap: an audible signal between the visual nudge and the full cascade could improve outcomes for residents who are still in the room but momentarily not paying attention.

4.3.3. Scenario C: Daytime adaptive lighting

Narrative. It is 14:00 on a weekday. The “Goedemorgen” routine has earlier set night_mode to off. Anna walks from the living room into the kitchen to make coffee, then back to the living room.

Automation flow. Living-room motion triggers “Woonkamer – licht adaptief aan”, which (with night_mode = off) turns on light.living_room_light at 80 % / 3500 K with a 3-second transition. Anna moves into the kitchen; the kitchen sensor triggers “Keuken – licht adaptief aan”, which turns on light.kitchen_light at 90 % / 4000 K and additionally turns on switch.woonkamerlamp so that the adjacent corner of the living room is also lit. Anna walks back to the living room; ten minutes after the kitchen sensor goes idle, “Keuken – licht uit na geen beweging” turns both the kitchen light and switch.woonkamerlamp off. Thirty minutes after the living-room sensor goes idle, “Woonkamer – licht uit na geen beweging” turns the living-room light off.

Observed behaviour. The colour-temperature shift between rooms was clearly perceptible: the kitchen at 4000 K felt deliberately task-oriented, while the living room at 3500 K felt softer. The 30-minute idle timer on the living room avoided the unpleasant pattern, common in motion-driven lighting, of the lights turning off while the resident is sitting still on the sofa. The kitchen's 10-minute timer also avoided the lights cutting out mid-cooking.

Result and discussion. The two characters of the system gentle warm at night, brighter and cooler during the day work as a single coherent lighting strategy across both modes. The same five lights serve both safety at night and orientation during the day, which is the result the design was aiming for. Daytime operation does not exercise voice triggers directly, but it depends on the morning "Goedemorgen" routine having transitioned the system back to its day character; this scenario therefore reads naturally as the second half of Scenario A.

4.3.4. Scenario D: Real-time night-mode transition

Narrative. Anna is in the bathroom at 21:55, before the scheduled night-mode activation at 22:00. The bathroom light is therefore on at 90 % / 4000 K. At 22:00, night_mode activates. The trigger is verified for all three input paths: the scheduled "Nachtmodus automatisch aan – weekdagen" automation, a press of the Zigbee night-mode button (Knop – nachtmodus schakelen), and the "Welterusten" Alexa routine.

Automation flow. All three triggering paths converge on the same state change of input_boolean.night_mode, which fires "Nachtmodus – actieve lichten aanpassen". The automation walks through light.bathroom_light, light.kitchen_light, light.living_room_light, and light.roomlamp, and for each one that is currently on, applies the night character (20 % / 2200 K) over a 10-second transition.

Observed behaviour. The bathroom light dimmed and warmed smoothly over ten seconds rather than stepping abruptly, regardless of which of the three input paths triggered the mode change. To Anna, the change felt natural rather than disruptive, the light "settled into night" rather than "switching modes."

Result and discussion. This convergence is what makes the input modalities feel coherent. The resident can switch the apartment into night mode by speaking, by pressing a button, or by simply waiting for the schedule, and in every case the visual experience is the same. From the resident's perspective, the choice of input is a matter of preference; from the system's perspective, the input layer is irrelevant once the state has changed.

4.3.5. Scenario E: Adaptive learning over time

Narrative. Over a two-week test period, the "Nachtmodus analyse elke week" automation runs every Monday at 03:05. Anna's actual sleep pattern, typical lights-out around 22:30 on weekdays and 23:30 on weekends, is recorded through the activation and deactivation events of input_boolean.night_mode, with most of those events triggered by the "Welterusten" and "Goedemorgen" Alexa routines rather than by the scheduled fallback.

Automation flow. Every weekday, "Nachtmodus automatisch aan/uit – weekdagen" reads input_datetime.night_mode_active_weekday_time and toggles night_mode accordingly, but only as a fallback if the resident hasn't already triggered the transition manually. The resident's actual transitions, regardless of source (Alexa routine, button press, scheduled trigger), are recorded in the Home Assistant history as state changes of input_boolean.night_mode. On Monday at 03:05, the Python analysis component receives the run_night_mode_analysis event, queries the history for the past week, computes a representative on time and off time for weekdays and weekend days separately, and writes the four resulting times into the four input_datetime helpers.

Observed behaviour. Over a two-week test period in which the simulated resident pattern shifted from a 22:00 / 06:00 default to a 22:30 / 06:30 effective pattern, driven mostly by "Welterusten" and "Goedemorgen" voice activations, the schedule converged to the observed times within two analysis cycles.

By the third week, the dashboard's auto-schedule entries reflected the actual sleep pattern without any manual configuration. A separate test compared a week of voice-driven activations against a week of button-driven activations: the analysis output was indistinguishable, with weekday on times within ten minutes of the configured target in both cases.

Result and discussion. The learning loop is input-agnostic by construction, which is why adding voice as a third input did not require any change to the analysis logic. Whether the resident says "Alexa, welterusten," presses the night-mode button, or simply lets the schedule do its work, the analysis component sees only the resulting state transition. The loop is also intentionally simple, one number for weekday on, one for weekday off, one for weekend on, one for weekend off, which makes it interpretable and auditable. A more sophisticated model (for example, context-aware bedtime that uses calendar or weather as input) is a natural next step for future research, and is discussed in Chapter 5; the current design makes that extension easy because it sits behind the same run_night_mode_analysis event interface.

4.4. Encountered limitations

Three practical limitations affected the project during development and testing. Three were resolved before the end of the project; one remains a structural constraint of the current platform.

4.4.1. Hardware capacity: migration from Raspberry Pi 3B to Pi 5

The prototype began on a Raspberry Pi 3B with 1 GB of RAM. As the project grew, adding Zigbee2MQTT, the Matter Hub add-on, the Python analysis component, the MQTT broker, and an increasing number of automations, the 3B reached its capacity ceiling. Symptoms included slow dashboard response, occasional automation execution delays, and general instability that made iterative development difficult. The system was migrated to a Raspberry Pi 5 with 8 GB of RAM by taking a full Home Assistant backup on the 3B and restoring it on the Pi 5. The migration preserved every entity, automation, history record, and dashboard; after the restore, the platform performed normally and has remained stable. This migration was a significant implementation decision, because it showed that even in a prototype environment, hardware capacity becomes a limiting factor once multiple integrations and automations run simultaneously. Future projects of similar scope should size the host platform for the expected integration count rather than starting at the minimum.

4.4.2. Network environment and IT-department dependencies

Since the project was implemented within a company setting, the Thomas More campus network, not all network-related decisions could be made directly inside the project. Connectivity, permissions, and static IP assignments depended partly on the IT department. This had a direct effect on testing and on the stability of certain connected devices. A concrete example was the Nuki Bridge: reliable operation depended on a static IP so that Home Assistant could always find the bridge after a network event or device restart. Until the reservation was arranged with the IT department, the Nuki integration produced intermittent failures that were difficult to reproduce. Once static IP reservations were in place for the Raspberry Pi, the Matter Hub host, and the Nuki Bridge, the entire class of failures disappeared. The project planning already recognized that infrastructure restrictions, compatibility issues, and permissions are relevant barriers in practical smart-home implementation; this experience confirms that assessment. Any future reproduction of the prototype in a similar institutional setting should resolve IP addressing and network permissions as a prerequisite, not as a late troubleshooting step.

4.4.3. Voice customization and Dutch language constraints

Two related limitations arose from the Alexa integration. The first concerns the language configuration of the Alexa device. When the Alexa app is configured in English, custom routine phrases can be created freely, including fully customized spoken responses, meaning it is possible to program Alexa to say any text the developer specifies. When the device language is set to Dutch, this customization is not available: Alexa can execute actions and play pre-defined content, but freely customized spoken responses are locked out. Because the prototype uses a Dutch-language Alexa to match the resident's language context, the “Welterusten” routine's spoken reply (“Good night. Talk to you tomorrow!”) and the “Alles goed” routine's “A-OK” are the pre-defined English phrases that Alexa's Dutch routine engine still permits, rather than fully custom Dutch text. The consequence is that the voice feedback the resident hears is in English within a Dutch-language interaction context, which is a usability inconsistency.

The second related limitation concerns voice recording. One of the originally explored features was having Alexa play a familiar voice recording, for example, a message recorded by a family member, during the “Bewustsense check” routine, so that the audible nudge at the check-in moment would feel personal and reassuring rather than impersonal. This was not achievable within the current Alexa skill and routine framework: Alexa's routine engine does not support playback of user-uploaded audio recordings as routine actions. The routine therefore plays a pre-selected Alexa song rather than a personalized message. This limitation is structural to the current platform and is unlikely to be resolved without a custom Alexa skill or an alternative voice platform; it is included as a research direction in Chapter 5.

4.5. Documentation deliverables

Three documents accompany the prototype alongside the project plan. Each targets a specific audience so that the system can be operated, reproduced, and reflected on by different stakeholders within Mobilab & Care.

- Project plan. Defines the objectives, business case, planning, and expected scope of the internship.
- User manual. Explains how a resident or demonstrator operates the system from the dashboard, the Zigbee buttons, and Alexa. It includes a reference to the installation process document for maintainers who need to reproduce or restore the system.
- Full installation process document. Explains how to reproduce the system from scratch: hardware preparation, Home Assistant Operating System installation, restoration from backup, Zigbee2MQTT pairing, Matter Hub setup, Nuki Bridge configuration, Telegram bot creation, CallMeBot registration, and Alexa routine setup.
- Reflection document. A personal account of the internship experience: what went well, what was difficult, what was learned, and what the author would approach differently in a future project.

4.6. Synthesis of testing

Across the five scenarios, the implemented prototype behaved as designed across all three input channels: motion, buttons, and voice. The night path operates as a coherent multi-zone flow rather than a sum of independent automations. The bathroom check-in and help-mode pipeline produces a humane safety chain that the resident can interact with through any of the three input paths: the timer chain provides a baseline that does not depend on the resident being conscious or reachable; “Alexa, alles goed” lets her cancel a false alarm without moving; “Alexa, hulpmodus” lets her request help without waiting for the timer chain; and the active Bewustsense check routine adds an audible nudge that reaches the resident even when she cannot see the bathroom light. Daytime operation reuses the same lighting infrastructure under a different character, and the morning Goedemorgen routine returns the system to that character in one phrase. The real-time light-adjustment automation eliminates abrupt mode transitions regardless of which input triggered them. The Python analysis loop is input-agnostic and converges to the resident’s actual rhythm whether transitions come from voice, buttons, or schedule. The encountered limitations, hardware capacity, network dependencies, and Dutch-language voice customisation constraints, are documented precisely enough that a future project can anticipate and mitigate them before they become blocking issues.

5. Conclusion

This internship set out to test, in practice, whether a locally controlled smart-home platform built on Home Assistant could provide a working, demonstrable, and reusable foundation for supporting safer night-time routines for older adults living alone in their own home. The realized prototype, developed and tested in the Experience Lab as a deliberate mimic of such a home, supports that thesis.

Three concrete contributions follow from the work. The first is a complete, working system rather than a partial implementation. Twenty-six automations are running; five Zigbee occupancy sensors and three buttons are paired and in use; five smart lights and one smart plug actuate the apartment; the Nuki smart lock is integrated as a first-class actuator that opens automatically during help-mode escalation; Telegram and WhatsApp are wired in as redundant notification channels; and Alexa controls the system over a local Matter Hub bridge. The total project-added hardware cost is approximately 680 €, well within the budget of an individual home.

The second contribution is that the prototype's behaviour is demonstrable through concrete scenarios. Anna's night-time bathroom trip, the no-return escalation, daytime adaptive lighting, the real-time night-mode transition, and the two-week adaptive-learning test each produce specific, observable outcomes that connect the technical implementation to the project's user-centred motivation. The same five lights serve both safety at night and orientation during the day; the bathroom check-in step is what makes the safety pipeline humane; the redundancy between Telegram and WhatsApp is what makes the alerting layer resilient; and the adaptive learning loop is what makes the schedule converge to the resident's actual rhythm without manual reconfiguration.

The third contribution is that the prototype is reproducible and extensible. The platform decisions favor local control, multi-vendor flexibility, and clear inspect ability through MQTT and the Home Assistant logbook. The hardware is inexpensive and off-the-shelf. The analysis loop is event-driven (run_night_mode_analysis), so more sophisticated learning components can be substituted for the current robust-statistic baseline without changing any other part of the system.

5.1. Future research directions

Several broader research directions naturally extend this prototype. They are listed not as concrete tasks for a successor internship, but as larger lines along which Mobilab & Care could build on the platform realized here.

Context-aware adaptation. The current adaptive component uses only the night-mode transition history. A richer model could integrate calendar information, weather data, recent activity inside the home, and external events to anticipate atypical days (a late evening visit, a hospital appointment the next morning, a heatwave that shifts the resident's sleep window). This is the natural place to introduce machine-learning techniques: not to replace the deterministic safety logic, but to make the schedule and ambient behavior genuinely context-sensitive.

Cross-system care integration. The Experience Lab already contains a Nobi fall-detection device. A future prototype could fuse Nobi's output, Home Assistant's motion history, and the bathroom-session state into a single care signal that distinguishes between routine quiet, anomalous quiet, and an actual incident. The same fusion approach extends naturally to integration with care-provider information systems, so that escalation decisions can take account of recent medical events or scheduled visits.

Multi-resident and household scenarios. The current logic implicitly assumes a single resident. Real homes increasingly include couples and informal carers who visit frequently. Differentiating between residents, for example through phone-based presence, voice identification, or pattern recognition on motion timing, is a non-trivial research direction, especially under privacy constraints, and would significantly broaden the population the system can serve.

Local voice and natural interaction. Voice control is currently provided by Alexa over a Matter bridge. As Home Assistant Voice and similar local-first stacks mature, the voice layer can move on-device, removing the last cloud dependency. Beyond merely replacing the platform, a research-oriented next step is to move from command-style interaction (“Alexa, hulpmodus aan”) to short conversational exchanges that can confirm intent, offer alternatives, or simply check in on the resident.

Wellness signals from ambient data. Two weeks of night-mode and motion history already contain rich information about sleep timing, night-time bathroom-trip frequency, and daily activity rhythms. Aggregated carefully and with explicit consent, these ambient signals could become a longitudinal wellness indicator that complements traditional clinical measurements. The privacy and ethical questions here are at least as important as the technical ones, and are themselves worth a dedicated study.

More broadly, the work confirms a position that the analysis chapter argued for and that the realization and testing chapters validated: a smart-home support system for older adults does not require expensive specialized hardware, cloud-tied platforms, or invasive monitoring. A small set of well-chosen sensors and lights, a local platform that exposes its own state, and automations designed around user-centred principles, gentle by default, escalation only when warranted, and routine-aware over time, are sufficient to produce a system that is both technically robust and humane in practice. That is the contribution this internship makes to the research direction of Mobilab & Care.

Reference list

Constantino, D. B., van der Veen, D. R., & Dijk, D.-J. (2025). The bright and dark side of blue-enriched light on sleep and activity in older adults. GeroScience. https://doi.org/10.1007/s11357-025-01506-y

Lu, X., Luo, Y., & Hu, B. (2022). Exploring older adults' nighttime trips to the bathroom under different lighting conditions: An exploratory field study. HERD: Health Environments Research & Design Journal. https://doi.org/10.1177/19375867221113067

Ratnayake, M., Somanath, P., & colleagues. (2022). Aging in place: Are we prepared? Delaware Journal of Public Health. https://pmc.ncbi.nlm.nih.gov/articles/PMC9495472/

Thölking, T. W., & colleagues. (2020). A guiding nightlight decreases fear of falling and increases sleep quality of community-dwelling older people: A quantitative and qualitative evaluation. Gerontology, 66(3), 295–303. https://doi.org/10.1159/000504883

Vaportzis, E., Giatsi Clausen, M., & Gow, A. J. (2017). Older adults' perceptions of technology and barriers to interacting with tablet computers: A focus group study. Frontiers in Psychology, 8, 1687. https://doi.org/10.3389/fpsyg.2017.01687

Walker, C. A., & Curry, L. C. (2007). Relocation stress syndrome in older adults transitioning from home to a long-term care facility: Myth or reality? Journal of Psychosocial Nursing and Mental Health Services, 45(1). https://pubmed.ncbi.nlm.nih.gov/17304985/

Wiles, J. L., Leibing, A., Guberman, N., Reeve, J., & Allen, R. E. (2012). The meaning of "aging in place" to older people. The Gerontologist, 52(3), 357–366. https://doi.org/10.1093/geront/gnr098

Apple. (n.d.). Over software-updates voor HomePod. Retrieved February 27, 2026, from https://support.apple.com/nl-nl/108045

Tweakers. (2024, August 23). Amazons Alexa ondersteunt voortaan Nederlands. https://tweakers.net/nieuws/225722/amazons-alexa-ondersteunt-voortaan-nederlands.html

Home Assistant. (n.d.). Home Assistant. Retrieved April 18, 2026, from https://www.home-assistant.io/

Home Assistant. (n.d.). MQTT. Retrieved April 18, 2026, from https://www.home-assistant.io/integrations/mqtt/

Home Assistant. (n.d.). Nuki Bridge. Retrieved April 18, 2026, from https://www.home-assistant.io/integrations/nuki/

Home Assistant. (n.d.). Getting started – Local [Voice control documentation]. Retrieved April 18, 2026, from https://www.home-assistant.io/voice_control/voice_remote_local_assistant/

Zigbee2MQTT. (n.d.). Zigbee2MQTT. Retrieved April 18, 2026, from https://www.zigbee2mqtt.io/

Home Assistant. (n.d.). Matter. Retrieved April 28, 2026, from https://www.home-assistant.io/integrations/matter/

Home Assistant. (n.d.). Telegram. Retrieved April 28, 2026, from https://www.home-assistant.io/integrations/telegram/

Home Assistant. (n.d.). Telegram bot. Retrieved April 28, 2026, from https://www.home-assistant.io/integrations/telegram_bot/

t0bst4r. (2026). Home-Assistant-Matter-Hub. GitHub repository. Retrieved April 28, 2026, from https://github.com/t0bst4r/home-assistant-matter-hub

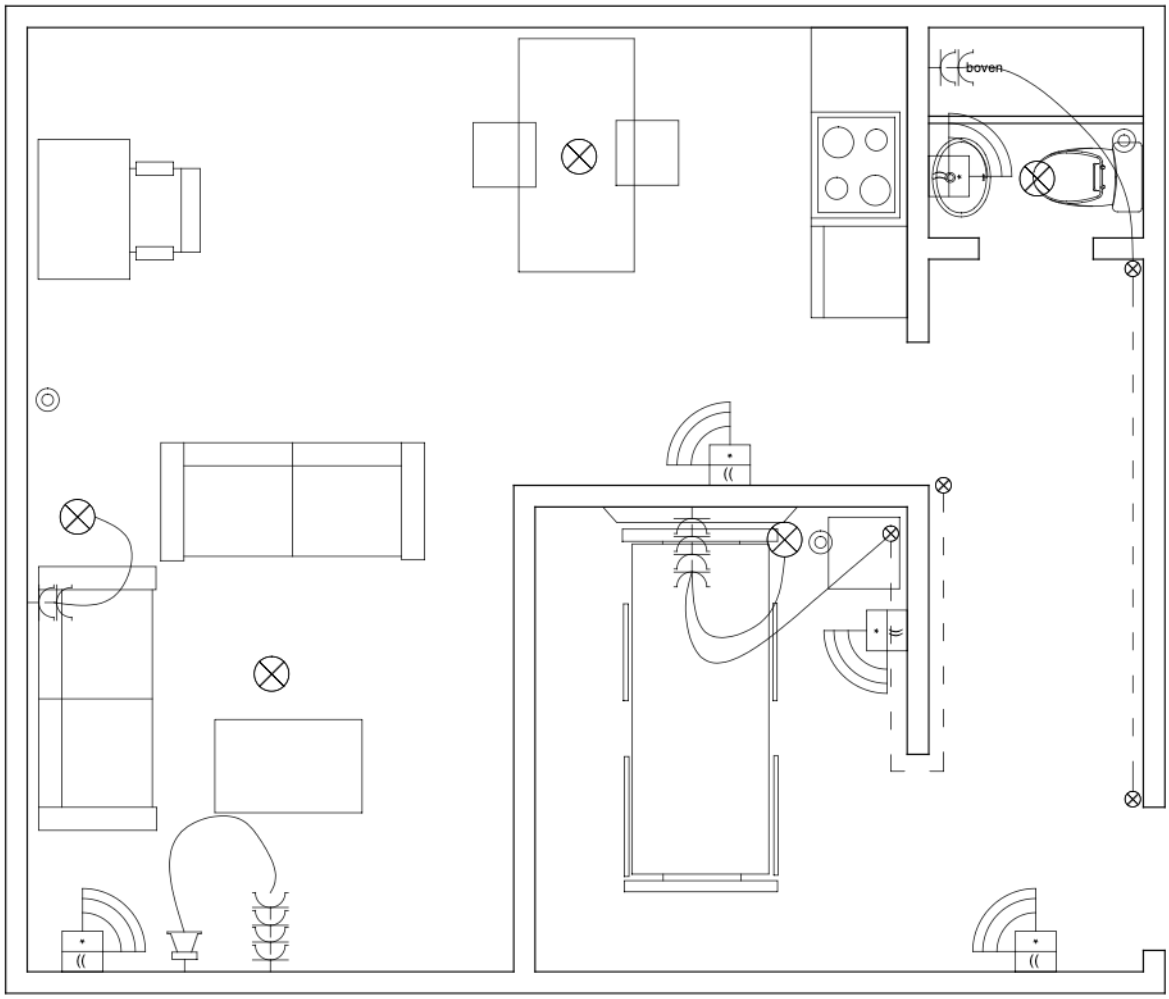
CallMeBot. (n.d.). Free API to Send WhatsApp Messages. Retrieved April 28, 2026, from https://www.callmebot.com/blog/free-api-whatsapp-messages/

Attachments

Appendix A. Experience Lab layout and control plan

Floor plan of the apartment and the simplified control/automation plan.

Figure A1. Simplified control plan indicating relevant smart-home zones, sensors, and lighting positions.



Appendix B. Visual impression of the lighting concept

Figure B1. Lumion visualization showing the intended visibility of the night path from the bedroom area.

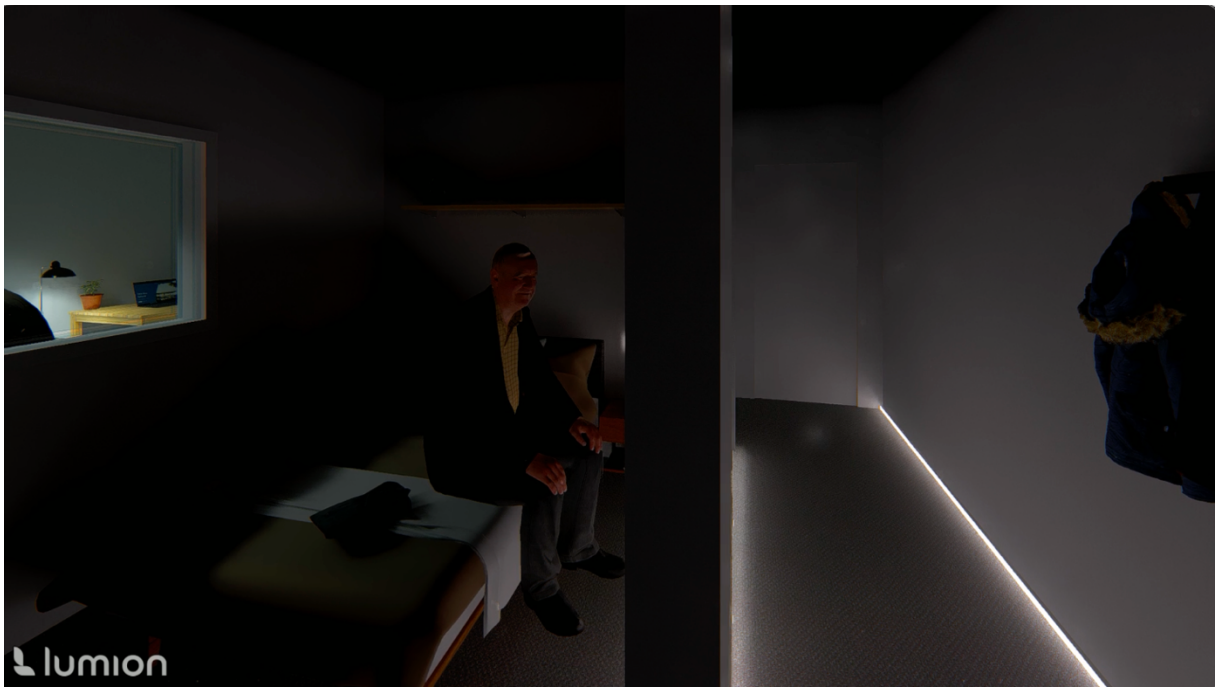


Figure B2. Lumion visualization of the corridor night-lighting effect.

Figure B3. Lumion visualization of the ambient lighting concept in the main living space.

Figure B4. Lumion visualization of the ambient lighting concept in the main kitchen space.